F21SF Software Engineering Foundations

Coursework Report

Auction House Inventory System

Student Name	Matriculation no
Pedakamsetti Harshitha	H00514683

Table of Contents

1. Key Feature Highlights 3

1.1 Unified Collectible Hierarchy 3

1.2 Robust CSV Data Loading and Validation 3

1.4 Interactive Swing User Interface 6

1.5 Safe Data Editing and Persistence 8

1.6 Comprehensive Error Handling 9

2. UML Class Diagram (Model Layer) 10

3. UML Activity Diagram 12

4. Testing Evidence 14

4.1 Unit Tests (JUnit) 14

4.2 System Test Matrix 14

4.3 Quality Assessment 15

5. Reflective Essay 15

6. Conclusion 17

1. Key Feature Highlights

This section provides a detailed overview of the six core features implemented in the Auction House Inventory System. Each feature is explained with its purpose, implementation approach, and how it benefits the user experience.

1.1 Unified Collectible Hierarchy

What it does: The system supports multiple types of collectibles (Furniture, Sculptures, and Coins) through an object-oriented inheritance structure.

How it works:

1. The abstract Collectible class defines common attributes shared by all collectibles: item ID, owner, condition, starting price, and year estimate.
2. Three concrete subclasses extend Collectible:
    a. Furniture: Adds furniture-specific fields (type, style, maker, length, height, depth)
    b. Sculpture: Adds sculpture-specific fields (subject, material, height)
    c. Coin: Adds coin-specific fields (material, origin, original value, currency)
3. Each subclass implements abstract methods (getCategory(), getShortDescriptor(), getLongDescription()) to provide type-specific descriptions.

Why it matters: This design allows the system to handle different collectible types uniformly while preserving their unique attributes. The inheritance hierarchy makes it easy to add new collectible types in the future without modifying existing code.

Code location: Collectible.java (abstract), Furniture.java, Sculpture.java, Coin.java (concrete subclasses)

1.2 Robust CSV Data Loading and Validation

What it does: The system reads collectible data from CSV files, automatically detects the collectible type, validates all fields, and handles corrupted or invalid data gracefully.

How it works:

1. **Type Detection:** The loadFromCsv() method reads each line and examines the first column to determine the collectible type (furniture, sculpture, or coin).
2. **Field Parsing:** Builder methods (buildFurniture(), buildSculpture(), buildCoin()) parse each field according to the collectible type's requirements.
3. **Validation:** Multiple validation checks ensure data integrity:
    a. **Column Count:** expectColumns() verifies each line has the correct number of fields (Furniture: 13, Sculpture: 10, Coin: 11)

   b. **Numeric Fields:** parseDouble() and parseInt() validate that numeric fields contain valid numbers, catching NumberFormatException and converting to clear error messages
   c. **Category Validation:** normaliseCondition() ensures condition values are from the allowed set (mint, restored, needs restoring)
   d. **Year Range:** YearEstimate constructor validates that low year ≤ high year
4. **Error Handling:** Invalid lines are caught, logged with line-specific error messages, and skipped. The program continues processing valid lines.

Why it matters: Users can work with real-world data files that may contain errors. The system identifies problems without crashing, allowing partial data loading and clear feedback about what needs to be fixed.

Code location: CollectibleMain.java

Screenshot reference:

```
========================================
Auction House Inventory System
========================================
Loading inventory from: collectibles_data.csv
Loaded items: 15

Inventory Summary
-----------------
Total items: 15
Unique owners: 12
Oldest span (low): Gold from Rome (SEST 200.00) (£11800.00) -> 60 (middle 68)
Newest span (high): Furniture stool by Design Within Reach (£150.00) -> 2020 (middle 2020)
Average price: £7160.67
Price standard deviation: £10435.89

Top spreads:
1. Furniture armchair by George Hepplewhite (£950.00) (1870-1890, spread 20)
2. Sculpture of Athena in Bronze (£42000.00) (430-450, spread 20)
3. Silver from Edinburgh (GBP 1.00) (£2200.00) (1690-1705, spread 15)

Statistics written to: inventory_statistics.txt
```

Fig 1: CLI Summary

Example error output:

Loading inventory from: collectibles_data.csv
Line 5: Expected 13 fields for furniture but found 12
Line 8: Invalid length: 'one hundred'
Loaded items: 15

Skipped lines due to validation errors: 2

1.3 Comprehensive Statistics and Reporting

What it does: The system generates detailed statistics about the inventory, including item counts, price analysis, year estimates, and condition breakdowns.

How it works:

1. **CLI Summary (Fig 1):** The printSummary() method displays a concise overview in the console, including:
    a. Total items and unique owner count
    b. Oldest item (based on low year estimate) with middle year shown
    c. Newest item (based on high year estimate) with middle year shown
    d. Average price and price standard deviation
    e. Top 3 items with largest year estimate spreads
2. **Detailed Report:** The generateStatisticsReport() method writes a comprehensive report to inventory_statistics.txt, including:
    a. All items from the CLI summary
    b. Most and least expensive items
    c. Condition breakdown (count of items per condition)
    d. Full details of top 3 year spreads
3. **Key methods used:**
4. findOldestByLowEstimate(): Finds item with lowest year estimate
5. findNewestByHighEstimate(): Finds item with highest year estimate
6. getTopSpreadItems(int count): Returns items with largest year estimate differences
7. getConditionBreakdown(): Groups items by condition and counts them
8. getAveragePrice(): Calculates mean starting price
9. getPriceStandardDeviation(): Calculates price variability

Why it matters: Users can quickly understand their inventory's characteristics, identify valuable items, and make informed decisions about pricing and cataloguing.

Code location: CollectibleInventory.java (statistics methods), collectibleMain.java (lines 191-228: printSummary())

Screenshot references:

```
========================================
AUCTION HOUSE INVENTORY STATISTICS
========================================

Total Items in Inventory: 15
Unique Owners Recorded: 12

Oldest Item (low estimate 60):
  Gold from Rome (SEST 200.00) (£11800.00)
  Middle Year Estimate: 68

Newest Item (high estimate 2020):
  Furniture stool by Design Within Reach (£150.00)
  Middle Year Estimate: 2020

Most Expensive Item (£42000.00):
  Sculpture of Athena in Bronze (£42000.00)

Least Expensive Item (£150.00):
  Furniture stool by Design Within Reach (£150.00)

Price Statistics:
  Average Price: £7160.67
  Standard Deviation: £10435.89

Condition Breakdown:
  mint: 8 items
  restored: 6 items
  needs restoring: 1 items

Top 3 Year Estimate Spreads:
  Furniture armchair by George Hepplewhite (£950.00)
    Low/High: 1870/1890 (spread 20)
  Sculpture of Athena in Bronze (£42000.00)
    Low/High: 430/450 (spread 20)
  Silver from Edinburgh (GBP 1.00) (£2200.00)
    Low/High: 1690/1705 (spread 15)

========================================
Report generated on: Fri Nov 21 13:43:57 GMT+04:00 2025
========================================
```

Fig 2: Statistical Summary

1.4 Interactive Swing User Interface

What it does: A graphical user interface allows users to view, search, sort, and manage collectibles without using the command line.

How it works:

1. **Main Table:** CollectibleTableModel displays all collectibles in a scrollable table with columns for:
    a. Item ID
    b. Category (Furniture/Sculpture/Coin)
    c. Short Descriptor (type-specific description)
    d. Owner
    e. Condition

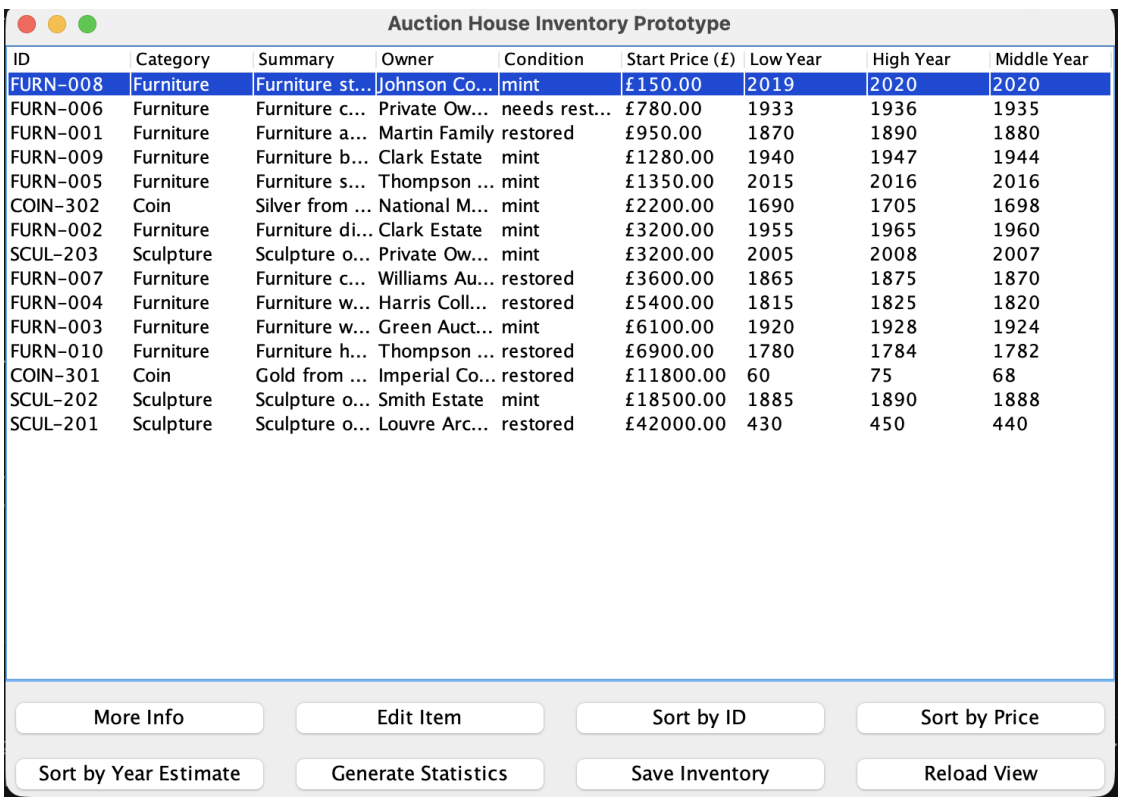
f. Starting Price
g. Low Year, High Year, Middle Year (from YearEstimate)

2. **Interactive Buttons:**
   a. **More Info:** Opens a dialog showing the full description (getLongDescription()) for the selected item
   b. **Edit Item:** Opens a dialog to modify price and condition (with validation)
   c. **Sort by ID / Price / Year Estimate:** Reorders the table by the selected criterion
   d. **Generate Statistics:** Refreshes the statistics report file
   e. **Save Inventory:** Writes all current data back to the CSV file
   f. **Reload View:** Refreshes the table display

Why it matters: The GUI makes the system accessible to non-technical users, providing an intuitive way to browse and manage inventory without learning command-line syntax.

Code location: InventoryAppFrame.java (main UI window), CollectibleTableModel.java (table data model)

Screenshot reference:



ID	Category	Summary	Owner	Condition	Start Price (£)	Low Year	High Year	Middle Year
FURN-008	Furniture	Furniture st...	Johnson Co...	mint	£150.00	2019	2020	2020
FURN-006	Furniture	Furniture c...	Private Ow...	needs rest...	£780.00	1933	1936	1935
FURN-001	Furniture	Furniture a...	Martin Family	restored	£950.00	1870	1890	1880
FURN-009	Furniture	Furniture b...	Clark Estate	mint	£1280.00	1940	1947	1944
FURN-005	Furniture	Furniture s...	Thompson ...	mint	£1350.00	2015	2016	2016
COIN-302	Coin	Silver from ...	National M...	mint	£2200.00	1690	1705	1698
FURN-002	Furniture	Furniture di...	Clark Estate	mint	£3200.00	1955	1965	1960
SCUL-203	Sculpture	Sculpture o...	Private Ow...	mint	£3200.00	2005	2008	2007
FURN-007	Furniture	Furniture c...	Williams Au...	restored	£3600.00	1865	1875	1870
FURN-004	Furniture	Furniture w...	Harris Coll...	restored	£5400.00	1815	1825	1820
FURN-003	Furniture	Furniture w...	Green Auct...	mint	£6100.00	1920	1928	1924
FURN-010	Furniture	Furniture h...	Thompson ...	restored	£6900.00	1780	1784	1782
COIN-301	Coin	Gold from ...	Imperial Co...	restored	£11800.00	60	75	68
SCUL-202	Sculpture	Sculpture o...	Smith Estate	mint	£18500.00	1885	1890	1888
SCUL-201	Sculpture	Sculpture o...	Louvre Arc...	restored	£42000.00	430	450	440

Fig 3: Swing User Interface

1.5 Safe Data Editing and Persistence

What it does: Users can edit collectible information through the UI, with validation to prevent invalid data entry, and save changes back to the CSV file.

How it works:

1. **Edit Dialog:** When "Edit Item" is clicked, a dialog appears with:
    a. Price field (validated as a positive number)
    b. Condition dropdown (restricted to valid options: mint, restored, needs restoring)
2. **Validation:**
    a. Price must be a valid positive number (non-numeric input shows an error dialog)
    b. Condition must be from the approved list
    c. Invalid inputs trigger friendly error dialogs instead of crashing
3. **Immediate Feedback:** After successful editing, the table refreshes automatically to show updated values
4. **Persistence:** "Save Inventory" button calls CollectibleInventory.saveToCsv(), which:
    a. Converts all in-memory Collectible objects back to CSV format
    b. Writes to the original CSV file (or a specified path)
    c. Preserves the exact CSV structure for compatibility

Why it matters: Users can correct data errors, update prices, and modify conditions without manually editing CSV files. All changes are validated and can be saved permanently.

Code location: InventoryAppFrame.java (lines 100-140: showEditDialog(), lines 142-150: saveInventory()), CollectibleInventory.java (lines 207-235: saveToCsv())

Screenshot references:

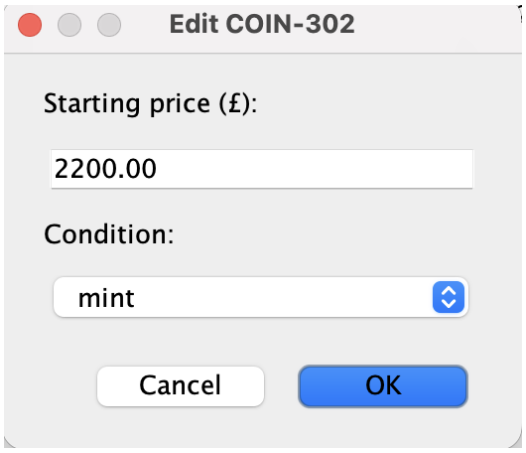


Fig 4: Edit Dialog

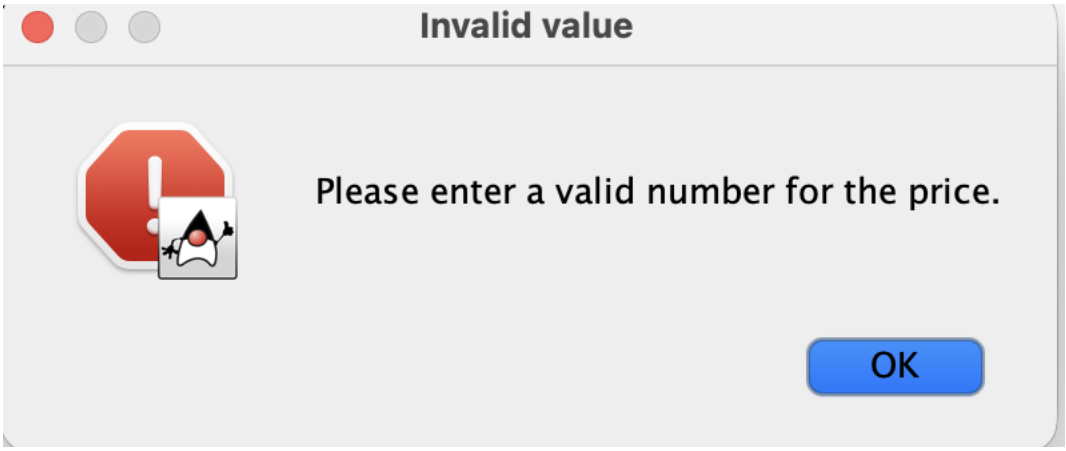


Fig 5: Validation

1.6 Comprehensive Error Handling

What it does: The system handles various error scenarios gracefully, providing clear feedback without crashing.

How it works:

1. **CSV Loading Errors:**
    a. Missing fields: Detected by expectColumns() method
    b. Non-numeric values: Caught by parseDouble()/parseInt() methods
    c. Invalid categories: Validated by normaliseCondition() and buildCollectible() methods
    d. Invalid year ranges: Validated by YearEstimate constructor
    e. All errors are caught in loadFromCsv() (lines 62-69), logged with line numbers, and the program continues
2. **File Errors:**
    a. Missing files: Clear error message "Unable to read file '...'" displayed
    b. Program exits gracefully without crashing
3. **UI Input Errors:**
    a. Invalid price: Error dialog shows "Invalid value" message
    b. Invalid condition: Error dialog explains allowed values
    c. No item selected: User-friendly prompt to select an item first

Why it matters: Robust error handling ensures the system remains usable even with problematic data. Users receive clear feedback about what went wrong and can take corrective action.

Code location: CollectibleMain.java (lines 49-76: CSV error handling), InventoryAppFrame.java (line 100-140: UI validation)

Screenshot reference:

```
Auction House Inventory System
========================================
Loading inventory from: collectibles_data_corrupted_missing.csv
Loaded items: 2
Skipped lines due to validation errors: 1

Inventory Summary
-----------------
Total items: 2
Unique owners: 2
Line 1: Expected 13 fields for furniture but found 12
Oldest span (low): Gold from Rome (SEST 250.00) (£16200.00) -> 40 (middle 45)
Newest span (high): Sculpture of Apollo in Bronze (£37500.00) -> 490 (middle 485)
Average price: £26850.00
Price standard deviation: £10650.00

Top spreads:
1. Sculpture of Apollo in Bronze (£37500.00) (480-490, spread 10)
2. Gold from Rome (SEST 250.00) (£16200.00) (40-50, spread 10)

Statistics written to: inventory_statistics.txt
```

Fig 6: Error Handling (collectibles_data_corrupted_missing)

2. UML Class Diagram (Model Layer)

What it shows:

- An abstract Collectible box with all the shared attributes and descriptor methods.
- Three subclasses (Furniture, Sculpture, Coin) linked with generalisation arrows.
- YearEstimate as a stand-alone value object used by every collectible.
- CollectibleInventory composed with the Collectible list, exposing the high-level operations the rest of the application relies on.

Design Assumptions and Decisions:

- **Inheritance over Composition:** We used an abstract Collectible class so all collectible types can be handled the same way, while each type keeps its unique features. Abstract methods ensure subclasses provide their own descriptions.
- **YearEstimate as Separate Class:** Instead of putting year fields directly in Collectible, we created a separate YearEstimate class. This keeps year data valid (low ≤ high) and makes calculations like spread easier. Once created, it can't be accidentally changed.
- **Inventory as Facade:** CollectibleInventory hides the internal list and only exposes high-level operations. This keeps the model separate from the UI, so the same code works with command-line, GUI, or web interfaces.

- **Model Independence:** The diagram shows only model classes, not UI classes. This makes it clear that the core logic can be reused in different interfaces without changes.

Why it matters: This diagram doubles as the knowledge transfer artifact. Anyone inheriting the project can see the responsibilities of each class, the direction of dependencies, and the deliberate choice to keep the UI out of the model layer.

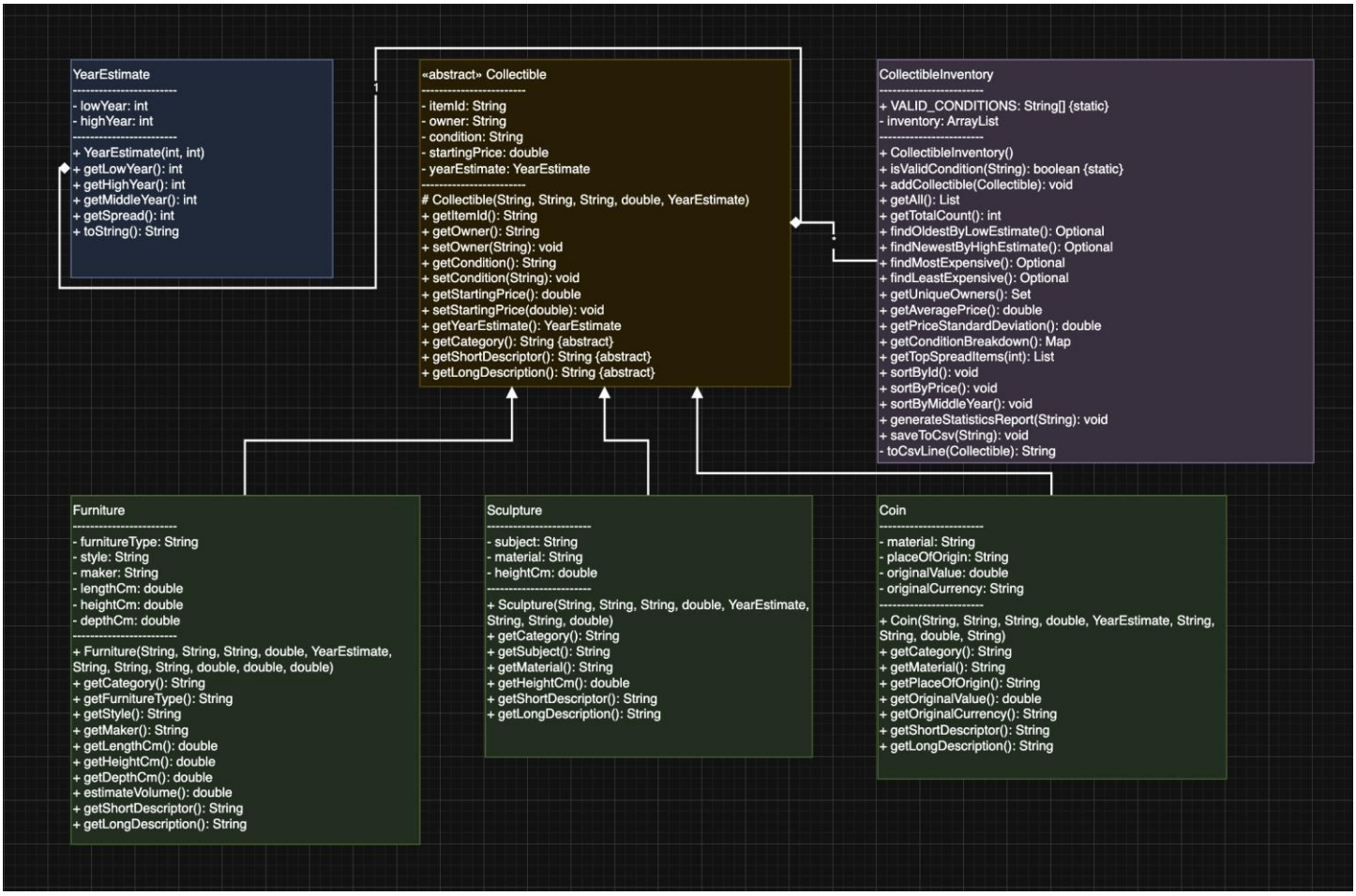


Fig 7: Class Diagram (Model Layer)

***Design Notes*:**

• Collectible is an abstract base class shared by all item types.
• Subclasses (Furniture, Sculpture, Coin) implement abstract methods and add type-specific attributes.
• YearEstimate is a value object (immutable) that encapsulates low/high year validation.
• CollectibleInventory manages a collection of Collectible objects and provides statistics.
• Composition: Collectible owns YearEstimate (lifecycle dependent).
• Aggregation: CollectibleInventory contains Collectible objects (can exist independently).

Legend:
Inheritance (Generalization): ————▶

Composition (strong ownership): —◆ (filled)
Aggregation (weak ownership): —◇ (hollow)

Visibility:
+ public
- private
protected
static
abstract

Color Coding:
Blue = Value Object (YearEstimate)
Yellow = Abstract Base (Collectible)
Green = Concrete Subclasses (Furniture, Sculpture, Coin)
Purple = Manager Class (CollectibleInventory)

3. UML Activity Diagram

- **Scenario:** "Compute the top three items with the largest difference between low and high year estimates."
- **Narrative:** The diagram mirrors the code path in CollectibleInventory.getTopSpreadItems(3). It starts when the statistics module requests the list, immediately handles the "empty inventory" branch, walks through the spread calculation loop, sorts the results, applies the limit, and returns the three most interesting items. This visual is useful during handovers because it's faster to digest than reading the method body.

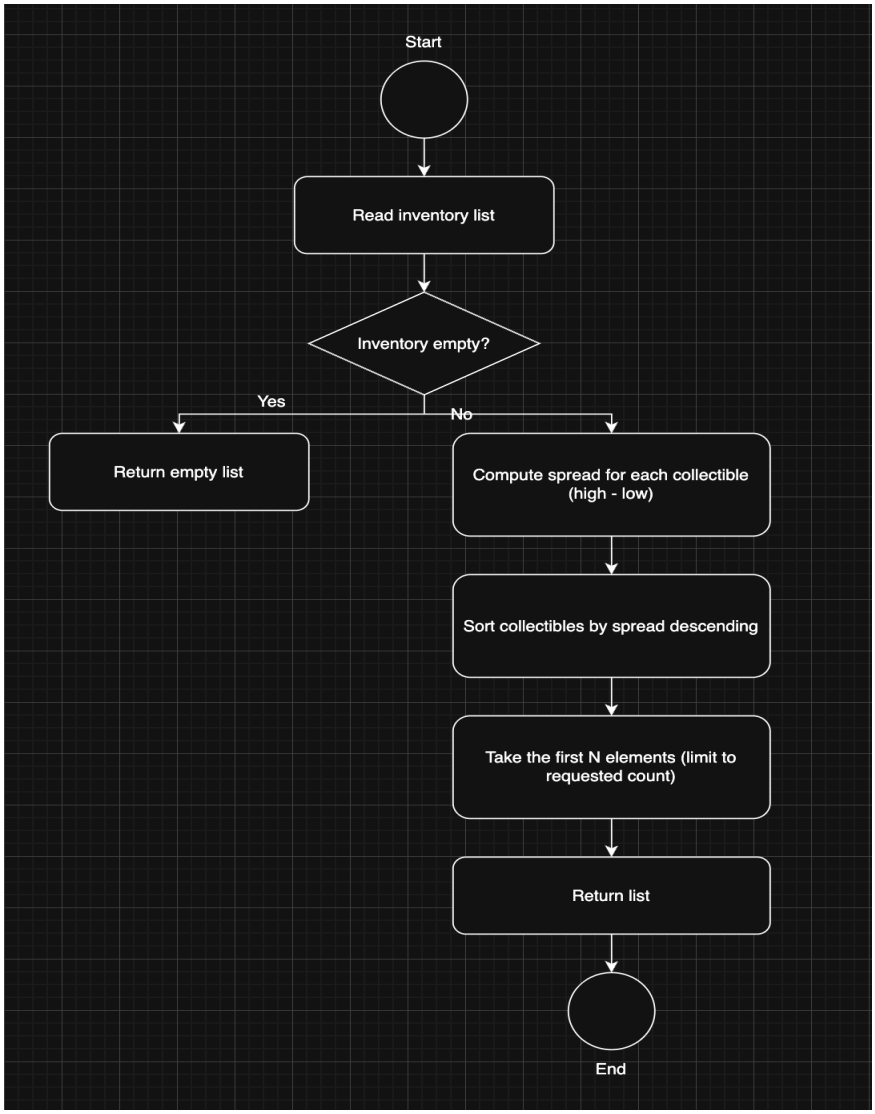


Fig 8: Activity Diagram (CollectibleInventory.getTopSpreadItems(3)

Shape Descriptions:
Start (Oval - Initial Node)
Read Inventory List (Rectangle - Activity)
Inventory Empty (Diamond - Decision)
Return Empty List (Rectangle - Activity)
Compute Spread for Each Collectible (Rectangle - Activity)
Sort Collectibles by Spread Descending (Rectangle - Activity)
Take the First N Elements (Rectangle - Activity)
Return List (Rectangle - Activity)
End (Oval - Final Node)

4. Testing Evidence

4.1 Unit Tests (JUnit)

Test name	Scenario	Expected / Actual
emptyInventoryReturnsEmptyList	No collectibles loaded	Returns empty list (no error)
singleItemReturnsSpreadZero	One item, low=high	Spread = 0, list size 1
ordersByDescendingSpread	Three spreads (20, 15, 5)	Order preserved (20→15→5)
limitsToRequestedCount	Five items, request top 3	Result size 3

4.2 System Test Matrix

IID	Action	Preconditions	Expected behaviour	Actual
1	Launch CLI + UI	collectibles_data.csv in project root	Console summary shows totals/owners/spreads; Swing window opens without errors	Pass
2	More Info dialog	Select Furniture row	Dialog lists every attribute (type, maker, dimensions, owner, condition, year estimate, price)	Pass
3	Edit item	Select a Coin, adjust price + condition	Table refreshes immediately, showing new price/condition; no validation error	Pass
4	Sort by Year Estimate	Items already loaded	Table entries reorder by computedmiddle year (oldest to newest)	Pass

5	Generate statistics	Inventory not empty	Modal confirms success; inventory_statistics.txt timestamp updates with new values	Pass
6	Save inventory	At least one edit pending	CSV file rewritten; rerunning program reflects edited values	Pass
7	Missing CSV	Rename input before launch	CLI prints "Unable to read file '...'" and exits without crashing the UI thread	Pass
8	Corrupted data files	Run with collectibles_data_corrupted_*.csv	Invalid lines skipped with line-specific error messages; valid lines still loaded; program continues normally	Pass

4.3 Quality Assessment

- **Strengths:** well-structured model layer, shared validation, statistics covered by unit tests, CSV output reflects edits.
- **Gaps:** no automated GUI tests; persistence still file-based and synchronous; console errors not surfaced in UI.
- **Next steps:** add parsing tests for malformed rows, introduce logging facade, explore GUI automation (e.g., AssertJ Swing) and lightweight database integration.

5. Reflective Essay

Waterfall, Agile, and DevOps: Critique and Relevance to Software Engineering Foundations

Software Engineering Foundations covers broad principles rather than a single specialism, so this reflection considers how each SDLC shapes coursework projects and future professional work.

Waterfall

Strengths: clear milestones, exhaustive documentation, predictable budgets. Ideal when university deliverables (e.g., architecture documents, class diagrams) must be approved before coding.

Weaknesses: slow feedback and limited tolerance for emerging insights. In Stage 3, a Waterfall gateway ensured the design (new subclasses, year estimate logic) was reviewed before coding, but mid-stage requirements (e.g., condition validation) required bending the plan, highlighting Waterfall's rigidity.

Agile

Strengths: iterative learning, stakeholder collaboration, visible progress. Weaknesses: risk of technical debt and scope creep if teams skip definition-of-done or refactoring. Within this module, I treated Stage 3 and Stage 4 as two sprints: backlog items (parser upgrade, statistics tweaks, Swing UI) were tackled incrementally, allowing rapid feedback from tutors. Agile practices (task boards, short retros) helped surface blockers early (e.g., CSV schema drift) and kept the codebase releasable at the end of each stage.

DevOps

Strengths: automation, shared responsibility between development and operations, continuous delivery mindset. Weaknesses: requires cultural buy-in and tooling maturity. Even in a coursework context, DevOps thinking influenced my approach: unit tests run before every refactor, CSV loading behaves deterministically, and statistics generation can be invoked via both CLI and UI, paving the way for automation. For future modules or internships, adopting CI pipelines (GitHub Actions, Jenkins) would turn these manual checks into enforced policies.

Comparative view

Aspect	Waterfall	Agile	DevOps
Planning	Heavy upfront, suits fixed briefs	Lightweight but continuous	Automates release planning through pipelines
Documentation	Formal specs and sign-offs	Living documentation, user stories	Runbooks, monitoring dashboards, Infrastructure as Code (IaC)
Feedback cycle	Long; often post-implementation	Short sprints, regular demos	Continuous (build/test/deploy feedback)
Fit for coursework	Great for design artefacts and grading checkpoints	Ideal for iterative coding tasks	Encourages disciplined testing and potential deployment demos

Personal practice

This project blended the three approaches: Waterfall ensured I delivered diagrams and documentation aligned with Stage goals; Agile techniques enabled rapid iterations between parsing logic and UI work; DevOps thinking kept me focused on repeatable tests and eventual automation. Going forward, I plan to formalize CI pipelines, add automated GUI tests, and document operational runbooks so the project can evolve into a fully DevOps-ready artifact. The key lesson is to select practices from each SDLC according to the project's phase and risk profile instead of following one methodology dogmatically.

6. Conclusion

The client's expanded requirements (multi-type collectibles, dual year estimates, richer statistics, Swing UI) are now satisfied and adds supporting documentation: UML diagrams, test artefacts, and reflective essays. Future improvements should target GUI automation, database-backed storage, and deployment-ready DevOps pipelines.